

CHƯƠNG 2

XÁC THỰC, PHÂN QUYỀN VÀ BẢO MẬT API

Thời lượng: 4 LT | 6 TH — Công nghệ: ASP.NET Core Identity, JWT, OAuth 2.0, Rate Limiting

MỤC TIÊU CHƯƠNG

Sau khi hoàn thành chương này, sinh viên có khả năng:

Về kiến thức

- Phân biệt rõ Authentication (xác thực danh tính) và Authorization (phân quyền truy cập); hiểu vì sao đây là hai lớp bảo mật độc lập.
- Giải thích cơ chế lưu trữ mật khẩu an toàn: tại sao không dùng MD5/SHA-1, PBKDF2 hoạt động như thế nào, và ASP.NET Core Identity cấu hình ra sao.
- Phân tích cấu trúc JWT (Header, Payload, Signature), hiểu cơ chế ký và xác minh, phân biệt HS256 và RS256.
- Trình bày kiến trúc Refresh Token Rotation — tại sao cần cặp Access Token + Refresh Token và cơ chế chống tấn công token theft.
- Hiểu các mô hình phân quyền: RBAC, Policy-Based Authorization và Resource-Based Authorization trong ASP.NET Core.
- Nắm vững Rate Limiting (giới hạn tần suất) và CORS (Cross-Origin Resource Sharing) — vai trò và cách cấu hình trong .NET 10.
- Hiểu OAuth 2.0 Authorization Code Flow với PKCE và cách tích hợp Social Login (Google) vào ứng dụng.

Về kỹ năng

- Cài đặt và cấu hình ASP.NET Core Identity với ApplicationUser tùy chỉnh.
- Implement đầy đủ luồng đăng ký, đăng nhập và refresh token với JWT trong .NET 10.
- Viết Authorization Policy và Resource-Based Authorization Handler bảo vệ đúng tài nguyên (recipe chỉ được sửa bởi tác giả).
- Cấu hình Rate Limiting (Fixed Window, Sliding Window) và CORS đúng cách cho Next.js frontend.
- Implement Social Login với Google sử dụng ASP.NET Core External Authentication.

Về thái độ

- Có tư duy "security-first" — bảo mật không phải tính năng thêm vào sau, mà phải được tích hợp từ đầu.

- Không lưu thông tin nhạy cảm (mật khẩu, secret) trong source code — luôn dùng environment variables hoặc Secret Manager.
- Hiểu và tôn trọng nguyên tắc Least Privilege: chỉ cấp quyền tối thiểu cần thiết.

GIỚI THIỆU CHƯƠNG

Bảo mật là một trong những khía cạnh quan trọng nhất — và thường bị xem nhẹ nhất — trong phát triển web. Theo báo cáo IBM Cost of a Data Breach 2024, chi phí trung bình của một vụ rò rỉ dữ liệu là 4,88 triệu USD. Phần lớn các vụ tấn công không đến từ lỗ hổng zero-day bí hiểm mà từ những sai lầm cơ bản: mật khẩu lưu dạng plaintext, JWT secret yếu, thiếu rate limiting trên endpoint đăng nhập.

Chương này xây dựng hệ thống bảo mật hoàn chỉnh cho Culinary Blog: từ việc lưu trữ mật khẩu an toàn, phát hành và xác minh JWT, quản lý phiên làm việc với Refresh Token Rotation, đến phân quyền chi tiết theo tác giả. Cuối chương, ứng dụng sẽ có một hệ thống Authentication/Authorization production-ready — các chương sau sẽ kế thừa nền tảng này.

Liên kết với Chương 1: Các Command/Query Handler đã được xây dựng ở Chương 1 sẽ được mở rộng với các lớp bảo mật. Infrastructure Layer sẽ được bổ sung Identity DbContext và JwtService. Presentation Layer sẽ thêm các Auth Endpoints và bảo vệ Recipe/Comment endpoints bằng JWT.

Bảo mật

Trong chương này, tất cả các đoạn code nhạy cảm (JWT key, client secret) đều được đọc từ cấu hình (IConfiguration) chứ không hardcode. Trong môi trường development, dùng dotnet user-secrets. Trong production, dùng Environment Variables hoặc cloud secret manager (Azure Key Vault, AWS Secrets Manager).

2.1. NỀN TẢNG BẢO MẬT WEB VÀ BĂM MẬT KHẨU

2.1.1 OWASP Top 10 — Tổng quan

OWASP (Open Web Application Security Project) công bố định kỳ danh sách 10 rủi ro bảo mật web phổ biến nhất. Thay vì học OWASP như một danh sách độc lập, mỗi mục sẽ được tích hợp trực tiếp vào chương liên quan. Bảng dưới đây là roadmap — chỉ ra chương nào trong giáo trình xử lý từng rủi ro.

OWASP 2021	Tên rủi ro	Chương xử lý chính
A01	Broken Access Control	Chương 2 — Resource-Based Authorization

OWASP 2021	Tên rủi ro	Chương xử lý chính
A02	Cryptographic Failures	Chương 2 — PBKDF2, JWT signing
A03	Injection (SQL, NoSQL, Command)	Chương 3 — EF Core parameterized queries
A04	Insecure Design	Chương 1 — Clean Architecture, threat modeling
A05	Security Misconfiguration	Chương 2 — CORS, HTTPS; Chương 6 — Docker security
A06	Vulnerable & Outdated Components	Chương 6 — CI/CD, dependency scanning
A07	Identification & Authentication Failures	Chương 2 — Mục tiêu chính của chương này
A08	Software & Data Integrity Failures	Chương 6 — GitHub Actions, image signing
A09	Security Logging & Monitoring Failures	Chương 6 — Serilog, OpenTelemetry
A10	Server-Side Request Forgery (SSRF)	Ghi chú trong Chương 4 — Next.js fetch

Bảng 2.1. OWASP Top 10 (2021) và chương tương ứng trong giáo trình

2.1.2 Tại sao không dùng MD5 hay SHA-256 để lưu mật khẩu?

Đây là một trong những lỗi bảo mật phổ biến nhất. MD5 và SHA-256 là các hàm hash một chiều — nhanh, nhưng đó chính là vấn đề. Một GPU hiện đại có thể tính khoảng 10 tỷ MD5 hashes mỗi giây. Kẻ tấn công có thể dùng Rainbow Table (bảng tra cứu mảng hash→password) hoặc brute-force toàn bộ 8-ký-tự passwords trong vài giờ.

Phương pháp lưu mật khẩu	Tốc độ hash	An toàn?	Lý do
Plaintext	N/A	✗ Không	Độc được trực tiếp — không cần crack
MD5 / SHA-1	~10B/s (GPU)	✗ Không	Quá nhanh, Rainbow Table, không có salt
SHA-256 + Salt	~1B/s (GPU)	⚠ Yếu	Nhanh hơn cần thiết, brute-force khả thi
PBKDF2 (100k vòng)	~1K/s (CPU)	☑ Tốt	Chậm có chủ ý, ASP.NET Core Identity dùng
BCrypt (cost=12)	~250/s (CPU)	☑ Tốt	Adaptive cost, phổ biến trong Node.js
Argon2id	~100/s (CPU)	☑ Tốt nhất	Winner PHC 2015, memory-hard, chống GPU

Bảng 2.2. So sánh các phương pháp lưu mật khẩu

2.1.3 PBKDF2 trong ASP.NET Core Identity

PBKDF2 (Password-Based Key Derivation Function 2) hoạt động bằng cách lặp lại hàm hash nhiều lần (iterations) trên password + salt ngẫu nhiên. Mỗi lần tăng gấp đôi số iterations, thời gian crack tăng gấp đôi — trong khi việc đăng nhập chỉ chậm hơn vài mili-giây. Đây là nguyên tắc "adaptive cost factor".

ASP.NET Core Identity mặc định sử dụng PBKDF2 với HMAC-SHA512. Từ .NET 6, số vòng lặp mặc định là 600.000 — phù hợp với khuyến nghị NIST 2023. Bạn có thể điều chỉnh qua PasswordHasherOptions.

Text CulinaryBlog.Infrastructure/DependencyInjection.cs (phần Identity)

```
// Cấu hình ASP.NET Core Identity với PBKDF2
builder.Services.Configure<PasswordHasherOptions>(options =>
{
    // Tăng số vòng Lặp PBKDF2 – mỗi lần tăng gấp đôi = tăng gấp đôi thời gian crack
    // 600_000 là khuyến nghị NIST 2023 cho PBKDF2-HMAC-SHA256
    // Với PBKDF2-HMAC-SHA512 (.NET default), có thể dùng 210_000 vòng
    options.IterationCount = 600000;
});

// Ví dụ sử dụng PasswordHasher trực tiếp (IPasswordHasher<T> được inject vào Handler)
// var hasher = new PasswordHasher<ApplicationUser>();
// string hash = hasher.HashPassword(user, "MyPassword123!");
// var result = hasher.VerifyHashedPassword(user, hash, "MyPassword123!");
// // result: Success, Failed, hoặc SuccessRehashNeeded (iterations cũ, cần hash lại)
```

Lưu ý

BCrypt và Argon2id không được ASP.NET Core Identity hỗ trợ native, nhưng bạn có thể implement custom IPasswordHasher<T> để dùng chúng. Trong giáo trình này, PBKDF2 của Identity là đủ tốt cho production. BCrypt/Argon2 được giới thiệu để sinh viên nhận biết khi làm việc với các stack khác (Node.js, Python).

2.2. JWT AUTHENTICATION

JSON Web Token (JWT) là tiêu chuẩn mở (RFC 7519) để truyền thông tin an toàn giữa các bên dưới dạng JSON object được ký số. JWT phù hợp với mô hình stateless API: server không cần lưu session — mọi thông tin cần thiết để xác thực đã nằm trong token. Điều này giúp API dễ scale ngang (horizontal scaling) vì bất kỳ server instance nào cũng có thể xác minh token.

2.2.1 Cấu trúc JWT

Một JWT gồm 3 phần phân cách bởi dấu chấm (.), mỗi phần được mã hóa Base64Url:



Hình 2.1. Cấu trúc JWT — Header, Payload và Signature (nhập vào Scalar để decode thực tế)

- Header: chứa loại token (JWT) và thuật toán ký (HS256, RS256). Base64Url encoded — KHÔNG phải mã hóa, ai cũng đọc được.
- Payload: chứa Claims — các cặp key/value mang thông tin về user và token. Base64Url encoded — cũng đọc được. KHÔNG lưu sensitive data như mật khẩu vào đây.
- Signature: phần DUY NHẤT được ký mật mã. Server dùng secret key để ký; khi nhận token, server ký lại và so sánh — nếu trùng thì token chưa bị giả mạo.

Bảo mật

JWT không mã hóa Payload — chỉ ký số để ngăn giả mạo. Bất kỳ ai có token đều đọc được nội dung bằng cách decode Base64Url. Điều này có nghĩa: (1) Không lưu mật khẩu, thông tin thẻ tín dụng hay dữ liệu nhạy cảm vào Payload; (2) HTTPS là bắt buộc để ngăn sniffing; (3) Nếu cần mã hóa Payload, dùng JWE (JSON Web Encryption).

2.2.2 Claims và các loại Claims

Claims là các "tuyên bố" về chủ thể (subject) của token. RFC 7519 định nghĩa một số Registered Claims chuẩn, ngoài ra có thể thêm Custom Claims.

Claim	Tên đầy đủ	Mô tả	Ví dụ trong Culinary Blog
sub	Subject	ID duy nhất của user	"sub": "3fa85f64-5717-..."
iss	Issuer	Server phát hành token	"iss": "https://api.culinaryblog.vn"
aud	Audience	Đối tượng nhận token	"aud": "culinaryblog-api"
exp	Expiration Time	Thời điểm token hết hạn	"exp": 1735689600 (Unix timestamp)
iat	Issued At	Thời điểm token được tạo	"iat": 1735686000
jti	JWT ID	ID duy nhất của token	"jti": "uuid-v4-..."
email	Custom	Email của user	"email": "chef@blog.vn"

Claim	Tên đầy đủ	Mô tả	Ví dụ trong Culinary Blog
role	Custom	Role của user	"role": "Author"

Bảng 2.3. Các loại Claims trong JWT

2.2.3 HS256 vs RS256 — Chọn thuật toán ký

Tiêu chí	HS256 (HMAC SHA-256)	RS256 (RSA SHA-256)
Loại key	Symmetric (1 secret key)	Asymmetric (private + public key)
Ai ký được	Bất kỳ ai có secret	Chỉ server có private key
Ai verify được	Chỉ ai có secret	Bất kỳ ai có public key
Phù hợp	Single service (monolith)	Microservices, multiple verifiers
Key distribution	Đơn giản	Phức tạp hơn (cần distribute public key)
Trong giáo trình	☒ Dùng trong Culinary Blog	Giới thiệu cho context microservices

Bảng 2.4. So sánh thuật toán ký HS256 và RS256

2.2.4 Implement JwtService trong .NET 10

Chúng ta tạo IJwtService interface trong Application Layer và JwtService implementation trong Infrastructure Layer — tuân thủ Dependency Inversion Principle đã học ở Chương 1.

C# CulinaryBlog.Application/Contracts/Services/IJwtService.cs

```
namespace CulinaryBlog.Application.Contracts.Services;

public interface IJwtService
{
    // Tạo Access Token từ thông tin user và danh sách roles
    string GenerateAccessToken(ApplicationUser user, IList<string> roles);

    // Tạo Refresh Token ngẫu nhiên, Lưu vào database
    RefreshToken GenerateRefreshToken(string userId);

    // Lấy claims từ Access Token đã hết hạn (dùng khi refresh)
    // Trả về null nếu token bị giả mạo (signature không hợp lệ)
    ClaimsPrincipal? GetPrincipalFromExpiredToken(string token);
}
```

C# CulinaryBlog.Domain/Settings/JwtSettings.cs

```
namespace CulinaryBlog.Domain.Settings;

// Configuration class – bind từ appsettings.json section "JwtSettings"
public class JwtSettings
{
    public const string SectionName = "JwtSettings";
}
```

```

// Secret key – tối thiểu 32 ký tự để đảm bảo entropy đủ cho HS256
public string Key { get; init; } = string.Empty;
public string Issuer { get; init; } = string.Empty;
public string Audience { get; init; } = string.Empty;
// Access Token ngắn hạn: 15 phút là best practice
public int AccessTokenExpiryMinutes { get; init; } = 15;
// Refresh Token dài hạn: 7 ngày
public int RefreshTokenExpiryDays { get; init; } = 7;
}

```

C# CulinaryBlog.Infrastructure.Services/JwtService.cs

```

using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;
using CulinaryBlog.Application.Contracts.Services;
using CulinaryBlog.Domain.Entities;
using CulinaryBlog.Domain.Settings;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;

namespace CulinaryBlog.Infrastructure.Services;

public class JwtService : IJwtService
{
    private readonly JwtSettings _settings;

    public JwtService(IOptions<JwtSettings> settings)
    => _settings = settings.Value;

    public string GenerateAccessToken(ApplicationUser user, IList<string> roles)
    {
        // Xây dựng danh sách Claims – thông tin được nhúng vào token
        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, user.Id),
            new(ClaimTypes.Email, user.Email!),
            new(ClaimTypes.Name, user.UserName!),
            new("fullName", user.FullName),
            // jti (JWT ID) – đảm bảo mỗi token là duy nhất
            new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
        };
        // Thêm tất cả roles vào Claims
        foreach (var role in roles)
            claims.Add(new Claim(ClaimTypes.Role, role));

        var key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _settings.Issuer,
            audience: _settings.Audience,
            claims: claims,
            notBefore: DateTime.UtcNow,
            expires: DateTime.UtcNow

```

```

.AddMinutes(_settings.AccessTokenExpiryMinutes),
    signingCredentials: creds);

    return new JwtSecurityTokenHandler().WriteToken(token);
}

public RefreshToken GenerateRefreshToken(string userId)
{
    // Tạo token ngẫu nhiên 64 bytes = 512 bits – đủ entropy, không thể đoán
    var tokenBytes = RandomNumberGenerator.GetBytes(64);
    return RefreshToken.Create(userId, Convert.ToBase64String(tokenBytes),
        _settings.RefreshTokenExpiryDays);
}

public ClaimsPrincipal? GetPrincipalFromExpiredToken(string token)
{
    var parameters = new TokenValidationParameters
    {
        ValidateIssuer          = true,
        ValidateAudience        = true,
        ValidateLifetime         = false, // QUAN TRỌNG: bỏ qua kiểm tra
        ValidateIssuerSigningKey = true,
        ValidIssuer              = _settings.Issuer,
        ValidAudience           = _settings.Audience,
        IssuerSigningKey         = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(_settings.Key)),
    };

    var handler = new JwtSecurityTokenHandler();
    var principal = handler.ValidateToken(token, parameters, out var
securityToken);

    // Chống "alg:none" attack – kẻ tấn công set algorithm thành none để
bypass signature
    if (securityToken is not JwtSecurityToken jwtToken ||
        !jwtToken.Header.Alg.Equals(SecurityAlgorithms.HmacSha256,
            StringComparison.InvariantCultureIgnoreCase))
    {
        throw new SecurityTokenException("Token không hợp lệ – algorithm
mismatch.");
    }

    return principal;
}
}

```

Text appsettings.json (phần cấu hình JWT)

```

{
  "JwtSettings": {
    "Key": "culinary-blog-super-secret-key-min-32-chars-
required!",
    "Issuer": "https://api.culinaryblog.vn",
    "Audience": "culinaryblog-client",
  }
}

```

```
"AccessTokenExpiryMinutes": 15,  
"RefreshTokenExpiryDays": 7  
}  
}
```

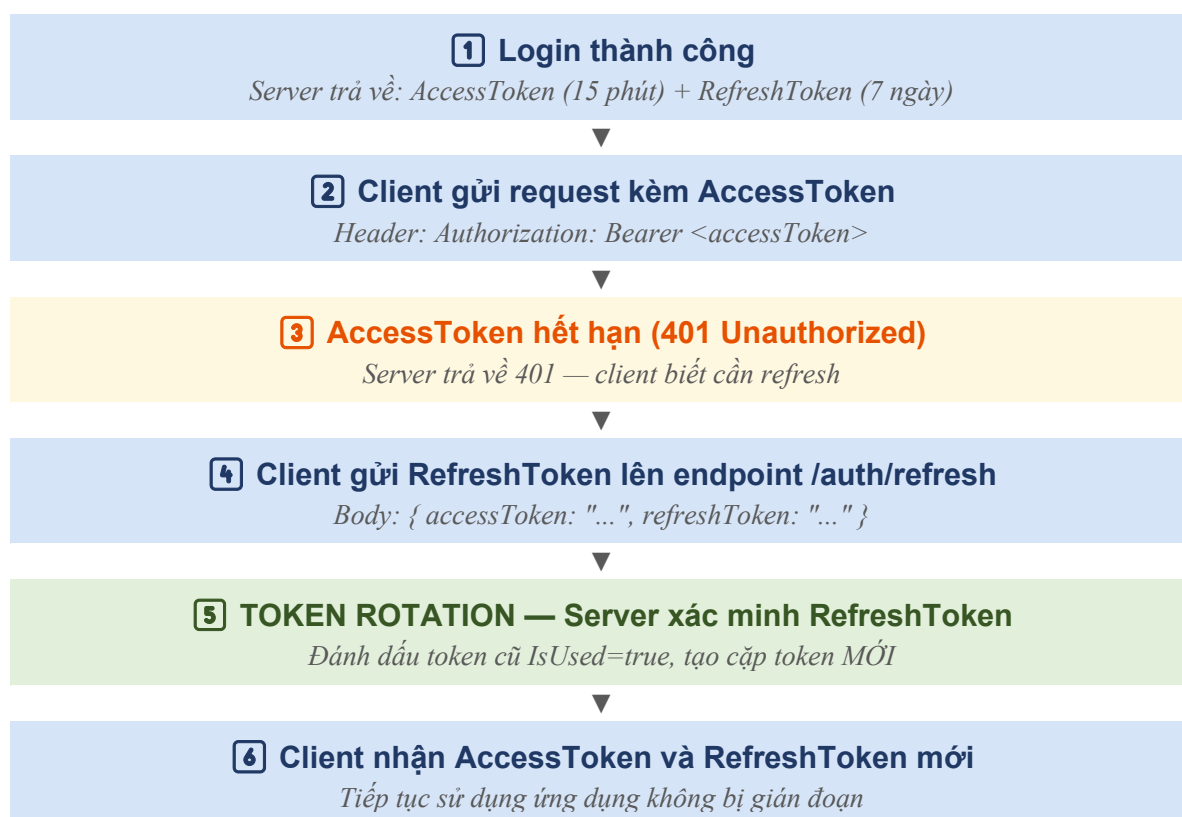
⚠ Cảnh báo

KHÔNG commit JwtSettings.Key vào Git repository. Dùng dotnet user-secrets trong development: `dotnet user-secrets set "JwtSettings:Key" "your-dev-secret"`. Trong production, đặt biến môi trường JWTSETTINGS__KEY hoặc dùng Azure Key Vault / AWS Secrets Manager.

2.3. REFRESH TOKEN ARCHITECTURE

Access Token có thời hạn ngắn (15 phút) để giảm thiểu rủi ro nếu bị đánh cắp. Nhưng nếu user phải đăng nhập lại mỗi 15 phút thì trải nghiệm rất tệ. Giải pháp: Refresh Token — một token dài hạn (7 ngày) được lưu server-side, dùng để phát hành Access Token mới mà không cần user nhập lại mật khẩu.

2.3.1 Luồng hoạt động của Refresh Token



Hình 2.2. Luồng hoạt động Refresh Token Rotation

Khái niệm

Token Rotation (Xoay vòng token): Mỗi lần dùng RefreshToken, server phát hành RefreshToken MỚI và đánh dấu token cũ là "đã dùng". Nếu kẻ tấn công đánh cắp RefreshToken và dùng trước user thực sự, khi user thực sự dùng token đó, server sẽ phát hiện nó đã bị dùng rồi → vô hiệu hóa TOÀN BỘ sessions của user và yêu cầu đăng nhập lại.

2.3.2 RefreshToken Entity và Repository

```
C# CulinaryBlog.Domain/Entities/RefreshToken.cs

namespace CulinaryBlog.Domain.Entities;

public class RefreshToken
{
    public Guid Id { get; private set; }
    // Token value - 64 bytes ngẫu nhiên, encode Base64
    public string Token { get; private set; } = string.Empty;
    public string UserId { get; private set; } = string.Empty;
    // Đã bị thu hồi chủ động (Logout, đổi mật khẩu, phát hiện bất thường)
    public bool IsRevoked { get; private set; }
    // Đã được dùng một lần (Token Rotation - không dùng lại được)
    public bool IsUsed { get; private set; }
    public DateTime ExpiresAt { get; private set; }
    public DateTime CreatedAt { get; private set; }

    // Navigation property đến ApplicationUser
    public ApplicationUser? User { get; private set; }

    protected RefreshToken() { }

    public static RefreshToken Create(string userId, string tokenValue, int expiryDays)
    {
        return new RefreshToken
        {
            Id = Guid.NewGuid(),
            Token = tokenValue,
            UserId = userId,
            ExpiresAt = DateTime.UtcNow.AddDays(expiryDays),
            CreatedAt = DateTime.UtcNow,
        };
    }

    // Kiểm tra token còn hợp lệ: chưa dùng, chưa thu hồi, chưa hết hạn
    public bool IsValid() => !IsRevoked && !IsUsed && ExpiresAt >
    DateTime.UtcNow;

    // Đánh dấu đã dùng sau Token Rotation
    public void MarkUsed() { IsUsed = true; }
    // Thu hồi khi user logout hoặc phát hiện bất thường
    public void Revoke() { IsRevoked = true; }
}
```

2.3.3 LoginCommand Handler — tạo JWT + RefreshToken

```
C# CulinaryBlog.Application/Features/Auth/Commands/Login/LoginCommandHandler.cs

using CulinaryBlog.Application.Common.Exceptions;
using CulinaryBlog.Application.Contracts.Persistence;
using CulinaryBlog.Application.Contracts.Services;
using CulinaryBlog.Application.DTOs;
using Mapster;
using MediatR;
using Microsoft.AspNetCore.Identity;

namespace CulinaryBlog.Application.Features.Auth.Commands.Login;

public class LoginCommandHandler : IRequestHandler<LoginCommand,
AuthResponseDto>
{
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly IJwtService _jwtService;
    private readonly IApplicationDbContext _context;

    public LoginCommandHandler(
        UserManager<ApplicationUser> userManager,
        IJwtService jwtService,
        IApplicationDbContext context)
    {
        _userManager = userManager;
        _jwtService = jwtService;
        _context = context;
    }

    public async Task<AuthResponseDto> Handle(
        LoginCommand command,
        CancellationToken cancellationToken)
    {
        // 1. Tìm user theo email (case-insensitive)
        var user = await _userManager.FindByEmailAsync(command.Email)
            ?? throw new UnauthorizedException("Email hoặc mật khẩu không chính
xác.");

        // 2. Kiểm tra tài khoản bị khóa (sau nhiều lần nhập sai)
        if (await _userManager.IsLockedOutAsync(user))
            throw new UnauthorizedException(
                "Tài khoản bị tạm khóa. Vui lòng thử lại sau 15 phút.");

        // 3. Xác minh mật khẩu dùng PBKDF2 (IPasswordHasher được inject vào
Identity)
        var isValid = await _userManager.CheckPasswordAsync(user,
command.Password);
        if (!isValid)
        {
            // Tăng failed login counter → kích hoạt lockout sau
MaxFailedAccessAttempts
            await _userManager.AccessFailedAsync(user);
            throw new UnauthorizedException("Email hoặc mật khẩu không chính
xác.");
        }

        // 4. Reset failed counter sau login thành công
        await _userManager.ResetAccessFailedCountAsync(user);
    }
}
```

```

// 5. Lấy danh sách roles
var roles = await _userManager.GetRolesAsync(user);

// 6. Tạo Access Token (short-lived, 15 phút)
var accessToken = _jwtService.GenerateAccessToken(user, roles);

// 7. Tạo Refresh Token (long-lived, 7 ngày), Lưu vào database
var refreshToken = _jwtService.GenerateRefreshToken(user.Id);
_context.RefreshTokens.Add(refreshToken);
await _context.SaveChangesAsync(cancellationToken);

return new AuthResponseDto(
    AccessToken:    accessToken,
    RefreshToken:   refreshToken.Token,
    AccessTokenExpiry: DateTime.UtcNow.AddMinutes(15),
    User:           user.Adapt<UserDto>());
}
}

```

2.3.4 RefreshTokenCommand — Token Rotation

```

C#
CulinaryBlog.Application/Features/Auth/Commands/RefreshToken/RefreshTokenCommandHandler.cs

public class RefreshTokenCommandHandler : IRequestHandler<RefreshTokenCommand,
AuthResponseDto>
{
    // ... constructor (UserManager, IJwtService, IApplicationDbContext)

    public async Task<AuthResponseDto> Handle(
        RefreshTokenCommand command,
        CancellationToken cancellationToken)
    {
        // 1. Xác minh chữ ký Access Token (bỏ qua thời hạn) – Lấy userId
        var principal = _jwtService.GetPrincipalFromExpiredToken(command.AccessToken)
            ?? throw new UnauthorizedException("Access token không hợp lệ.");

        var userId = principal.FindFirst(ClaimTypes.NameIdentifier)?.Value
            ?? throw new UnauthorizedException("Token không chứa user ID.");

        // 2. Tìm Refresh Token trong database, đảm bảo khớp với userId
        var storedToken = await _context.RefreshTokens
            .FirstOrDefaultAsync(rt =>
                rt.Token == command.RefreshToken &&
                rt.UserId == userId,
                cancellationToken)
            ?? throw new UnauthorizedException("Refresh token không tồn tại.");

        // 3. Kiểm tra token có còn hợp lệ
        if (!storedToken.IsValid())
        {
            // Phát hiện token đã bị dùng → có thể token bị đánh cắp
            // Thu hồi TOÀN BỘ tokens của user và yêu cầu đăng nhập lại
            if (storedToken.IsUsed)
            {
                var allTokens = await _context.RefreshTokens

```

```

        .Where(rt => rt.UserId == userId).ToListAsync(cancellationToken);
        allTokens.ForEach(t => t.Revoke());
        await _context.SaveChangesAsync(cancellationToken);
    }
    throw new UnauthorizedException("Refresh token đã hết hạn hoặc đã được sử dụng.");
}

// 4. Tìm user
var user = await _userManager.FindByIdAsync(userId)!;
var roles = await _userManager.GetRolesAsync(user);

// 5. TOKEN ROTATION – đánh dấu token cũ đã dùng
storedToken.MarkUsed();

// 6. Tạo cặp token mới
var newAccessToken = _jwtService.GenerateAccessToken(user, roles);
var newRefreshToken = _jwtService.GenerateRefreshToken(userId);
_context.RefreshTokens.Add(newRefreshToken);
await _context.SaveChangesAsync(cancellationToken);

return new AuthResponseDto(
    AccessToken: newAccessToken,
    RefreshToken: newRefreshToken.Token,
    AccessTokenExpiry: DateTime.UtcNow.AddMinutes(15),
    User: user.Adapt<UserDto>());
}
}

```

2.4. AUTHORIZATION MODELS

Authentication (Xác thực) trả lời câu hỏi "Bạn là ai?" — dùng JWT để xác định identity. Authorization (Phân quyền) trả lời câu hỏi "Bạn được phép làm gì?" — là bước hoàn toàn tách biệt xảy ra SAU Authentication. ASP.NET Core có ba mô hình phân quyền từ đơn giản đến phức tạp.

2.4.1 Role-Based Access Control (RBAC)

RBAC gán quyền dựa trên Role của user. Role được lưu trong JWT Claims nên không cần truy vấn database mỗi request — đây là ưu điểm lớn của JWT. Culinary Blog có 3 roles: Admin, Author, Reader.

Text CulinaryBlog.API/Endpoints/AdminEndpoints.cs (RBAC example)

```

// RBAC với Minimal APIs: yêu cầu role "Admin"
app.MapGet("/api/v1/admin/users", async (ISender sender) =>
{
    var users = await sender.Send(new GetAllUsersQuery());
    return Results.Ok(users);
})
.RequireAuthorization("AdminOnly"); // Tên policy

```

```
// Định nghĩa policy "AdminOnly" trong DependencyInjection.cs
services.AddAuthorizationBuilder()
    .AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin")) // Role phải có trong JWT claims
    .AddPolicy("AuthorOrAdmin", policy =>
        policy.RequireRole("Author", "Admin"))
    .AddPolicy("AuthenticatedUser", policy =>
        policy.RequireAuthenticatedUser());
```

2.4.2 Policy-Based Authorization

Policy-Based Authorization cho phép định nghĩa các điều kiện phức tạp hơn Role — ví dụ: user phải có email xác nhận VÀ đã đăng ký hơn 30 ngày. Mỗi Policy gồm một hoặc nhiều Requirements và Handlers tương ứng.

```
C# CulinaryBlog.Infrastructure.Authorization/MinimumAgeRequirement.cs

using Microsoft.AspNetCore.Authorization;

// Requirement: định nghĩa điều kiện cần kiểm tra
public class VerifiedAuthorRequirement : IAuthorizationRequirement
{
    // User phải có email đã xác nhận để publish bài viết
}

// Handler: chứa logic kiểm tra
public class VerifiedAuthorHandler
    : AuthorizationHandler<VerifiedAuthorRequirement>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        VerifiedAuthorRequirement requirement)
    {
        // Lấy claim "email_confirmed" từ JWT
        var emailConfirmed = context.User
            .FindFirst("email_confirmed")?.Value;

        if (emailConfirmed == "true")
            context.Succeed(requirement);
        else
            context.Fail(new AuthorizationFailureReason(this,
                "Cần xác nhận email trước khi đăng bài."));

        return Task.CompletedTask;
    }
}

// Đăng ký trong DI:
// services.AddSingleton<IAuthorizationHandler, VerifiedAuthorHandler>();
// builder.AddPolicy("VerifiedAuthor", p => p.Requirements.Add(new
// VerifiedAuthorRequirement()));
```

2.4.3 Resource-Based Authorization — Owner Authorization

Resource-Based Authorization là mô hình phức tạp nhất: kiểm tra quyền dựa trên cả identity của user VÀ thuộc tính của resource cụ thể. Trong Culinary Blog: chỉ tác giả (AuthorId == user.Id) mới được sửa/xóa recipe của mình. Đây không thể làm với Role hay Policy đơn thuần vì cần biết recipe đang được thao tác là của ai.

```
C# CulinaryBlog.Infrastructure.Authorization.RecipeAuthorizationHandler.cs

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authorization.Infrastructure;
using System.Security.Claims;

// Dùng OperationAuthorizationRequirement có sẵn: Create, Read, Update, Delete
public class RecipeAuthorizationHandler
    : AuthorizationHandler<OperationAuthorizationRequirement, Recipe>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        OperationAuthorizationRequirement requirement,
        Recipe recipe) // Resource cụ thể
    {
        var userId = context.User
            .FindFirst(ClaimTypes.NameIdentifier)?.Value;

        // Admin có toàn quyền
        if (context.User.IsInRole("Admin"))
        {
            context.Succeed(requirement);
            return Task.CompletedTask;
        }

        // Read: mọi người đều đọc được (recipe public)
        if (requirement.Name == Operations.Read.Name)
        {
            context.Succeed(requirement);
            return Task.CompletedTask;
        }

        // Update, Delete: chỉ tác giả
        if ((requirement.Name == Operations.Update.Name ||
            requirement.Name == Operations.Delete.Name) &&
            userId == recipe.AuthorId.ToString())
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}

// Hằng số cho các operations – tránh magic strings
public static class Operations
{
    public static readonly OperationAuthorizationRequirement Create = new() {
        Name = "Create" };
    public static readonly OperationAuthorizationRequirement Read = new() {
        Name = "Read" };
}
```

```

    public static readonly OperationAuthorizationRequirement Update = new() {
        Name = "Update" };
    public static readonly OperationAuthorizationRequirement Delete = new() {
        Name = "Delete" };
}

```

Text CulinaryBlog.API/Endpoints/RecipeEndpoints.cs (Authorization usage)

```

// Dùng IAuthorizationService trong Endpoint để kiểm tra Resource-Based
Authorization
app.MapDelete("/api/v1/recipes/{id:guid}", async (
    Guid id,
    ISender sender,
    IAuthorizationService authorizationService,
    ClaimsPrincipal currentUser,
    IApplicationDbContext dbContext,
    CancellationToken ct) =>
{
    // 1. Lấy recipe từ database
    var recipe = await dbContext.Recipes
        .FindAsync([id], ct);
    if (recipe is null) return Results.NotFound();

    // 2. Kiểm tra quyền xóa – Resource-Based Authorization
    var authResult = await authorizationService.AuthorizeAsync(
        currentUser, recipe, Operations.Delete);

    if (!authResult.Succeeded)
        return Results.Forbid(); // 403 Forbidden

    // 3. Xóa recipe
    await sender.Send(new DeleteRecipeCommand(id), ct);
    return Results.NoContent(); // 204
})
.RequireAuthorization(); // Phải đăng nhập (authenticated user)

```

2.5. RATE LIMITING VÀ CORS

2.5.1 Rate Limiting — Giới hạn tần suất request

Rate Limiting bảo vệ API khỏi các cuộc tấn công brute-force (thử mật khẩu hàng nghìn lần/phút) và DDoS (làm ngập server bằng requests). Từ .NET 7, ASP.NET Core có Rate Limiting middleware tích hợp sẵn — không cần thư viện bên ngoài.

Thuật toán	Cơ chế	Phù hợp với	Nhược điểm
Fixed Window	X requests trong cửa sổ cố định (VD: 5 req/phút)	Login endpoint, simple API	Burst tại ranh giới cửa sổ
Sliding Window	X requests trong cửa sổ trượt liên tục	Công bằng hơn, chống burst	Tốn bộ nhớ hơn

Thuật toán	Cơ chế	Phù hợp với	Nhược điểm
Token Bucket	Token tích lũy theo thời gian, mỗi request dùng 1 token	Cho phép burst ngắn, API chung	Phức tạp hơn
Concurrency	Giới hạn số request xử lý đồng thời (không phải/thời gian)	CPU-intensive endpoints	Không chống brute-force

Bảng 2.5. So sánh các thuật toán Rate Limiting

Text CulinaryBlog.Infrastructure/DependencyInjection.cs (Rate Limiting)

```

public static IServiceCollection AddInfrastructure(
    this IServiceCollection services, IConfiguration configuration)
{
    // ... (EF Core, Identity setup ở trên)

    // — Rate Limiting
    services.AddRateLimiter(options =>
    {
        // Giới hạn nghiêm ngặt cho Login: 5 attempts/phút
        // Mục đích: chống brute-force mật khẩu
        options.AddFixedWindowLimiter("login-policy", opt =>
        {
            opt.Window      = TimeSpan.FromMinutes(1);
            opt.PermitLimit = 5;
            opt.QueueLimit  = 0; // Không queue – reject ngay nếu vượt hạn
        });

        // Rate limit chung cho API: 200 requests/phút/client
        options.AddTokenBucketLimiter("api-policy", opt =>
        {
            opt.TokenLimit      = 200;
            opt.TokensPerPeriod = 200;
            opt.ReplenishmentPeriod = TimeSpan.FromMinutes(1);
            opt.AutoReplenishment = true;
        });

        // Trả về 429 Too Many Requests (không dùng 503)
        options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

        // Custom response khi bị rate limited
        options.OnRejected = async (context, ct) =>
        {
            context.HttpContext.Response.Headers["Retry-After"] = "60";
            await context.HttpContext.Response.WriteAsJsonAsync(new
            {
                type: "https://tools.ietf.org/html/rfc6585#section-4",
                title: "Quá nhiều yêu cầu.",
                status: 429,
                detail: "Bạn đã gửi quá nhiều yêu cầu. Vui lòng thử lại sau 60
giây.",
            }, ct);
        });
    });
}

```

```
    return services;
}
```

2.5.2 CORS — Cross-Origin Resource Sharing

Same-Origin Policy là cơ chế bảo mật của browser: script chạy trên origin A (ví dụ: `http://localhost:3000`) không được phép gửi request đến origin B (`http://localhost:5001`) mà không có phép. CORS là cơ chế server "nói lỏng" Same-Origin Policy có kiểm soát: server thông báo cho browser biết origin nào được phép gửi request.

Lưu ý: CORS CHỈ áp dụng cho browser — không bảo vệ khỏi request từ server, Postman hay curl. CORS không phải là biện pháp bảo mật API — đó là Authentication và Authorization.

```
Text CulinaryBlog.Infrastructure/DependencyInjection.cs (CORS)

services.AddCors(options =>
{
    // Policy cho môi trường Development
    options.AddPolicy("DevelopmentPolicy", policy =>
    {
        policy
            .WithOrigins("http://localhost:3000") // Next.js dev server
            .WithMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
            .WithHeaders("Authorization", "Content-Type", "Accept", "X-Request-
ID")
            .AllowCredentials(); // Cần nếu dùng httpOnly cookies
    });

    // Policy cho Production
    options.AddPolicy("ProductionPolicy", policy =>
    {
        policy
            .WithOrigins(
                "https://culinaryblog.vn",
                "https://www.culinaryblog.vn")
            .WithMethods("GET", "POST", "PUT", "PATCH", "DELETE")
            .WithHeaders("Authorization", "Content-Type")
            .AllowCredentials()
            .SetPreflightMaxAge(TimeSpan.FromMinutes(10)); // Cache preflight 10
phút
    });
});

// Trong Program.cs, áp dụng đúng policy theo môi trường:
// var corsPolicy = app.Environment.IsDevelopment()
//     ? "DevelopmentPolicy" : "ProductionPolicy";
// app.UseCors(corsPolicy);
```

2.6. OAUTH 2.0 VÀ OPENID CONNECT

OAuth 2.0 (RFC 6749) là framework ủy quyền cho phép ứng dụng của bên thứ ba truy cập tài nguyên của user mà không cần biết mật khẩu. OpenID Connect (OIDC) là tầng identity xây dựng trên OAuth 2.0: trong khi OAuth 2.0 xử lý authorization (cấp phép), OIDC xử lý authentication (xác thực danh tính user).

2.6.1 Authorization Code Flow với PKCE

Đây là flow được khuyến nghị cho web apps và mobile apps. PKCE (Proof Key for Code Exchange) thêm một lớp bảo vệ chống tấn công "authorization code interception" — đặc biệt quan trọng với SPA và mobile apps không thể giữ client_secret an toàn.



Hình 2.3. OAuth 2.0 Authorization Code Flow với PKCE cho Social Login

2.6.2 Implement Google Social Login

Text CulinaryBlog.Infrastructure/DependencyInjection.cs (Google Auth)

```
// Thêm Google Authentication provider
services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options => { /* ... JWT config như trên */ })
```

```
.AddGoogle(googleOptions =>
{
    googleOptions.ClientId = configuration["Google:ClientId"]!;
    googleOptions.ClientSecret = configuration["Google:ClientSecret"]!;
    // Callback URL phải khớp với Google Cloud Console configuration
    googleOptions.CallbackPath = "/api/v1/auth/google/callback";
    // Lấy thêm profile picture
    googleOptions.Scope.Add("profile");
});
```

```
C#
CulinaryBlog.Application/Features/Auth/Commands/GoogleLogin/GoogleLoginCommandHandler.cs

public class GoogleLoginCommandHandler : IRequestHandler<GoogleLoginCommand,
AuthResponseDto>
{
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly IJwtService _jwtService;
    private readonly IApplicationDbContext _context;

    public async Task<AuthResponseDto> Handle(
        GoogleLoginCommand command, CancellationToken ct)
    {
        // Thông tin từ Google được ASP.NET Core tự động verify và parse
        // command.ExternalLoginInfo chứa: email, name, picture từ Google token

        // 1. Tìm user theo external login (Google Provider + Provider Key)
        var user = await _userManager.FindByLoginAsync(
            command.ExternalLoginInfo.LoginProvider,
            command.ExternalLoginInfo.ProviderKey);

        // 2. Nếu chưa có, tạo account mới từ thông tin Google
        if (user is null)
        {
            var email = command.ExternalLoginInfo.Principal
                .FindFirst(ClaimTypes.Email)?.Value!;

            // Kiểm tra email đã tồn tại chưa (có thể đăng ký thủ công trước)
            user = await _userManager.FindByEmailAsync(email);

            if (user is null)
            {
                // Tạo user mới từ Google profile
                user = ApplicationUser.CreateFromGoogle(
                    email,

                    command.ExternalLoginInfo.Principal.FindFirst(ClaimTypes.Name)?.Value ?? email,

                    command.ExternalLoginInfo.Principal.FindFirst("picture)?.Value);

                await _userManager.CreateAsync(user);
                await _userManager.AddToRoleAsync(user, "Author"); // Default role
            }

            // Liên kết Google login với user account
            await _userManager.AddLoginAsync(user, command.ExternalLoginInfo);
        }
    }
}
```

```

var roles      = await _userManager.GetRolesAsync(user);
var accessToken = _jwtService.GenerateAccessToken(user, roles);
var refreshToken = _jwtService.GenerateRefreshToken(user.Id);

_context.RefreshTokens.Add(refreshToken);
await _context.SaveChangesAsync(ct);

return new AuthResponseDto(accessToken, refreshToken.Token,
    DateTime.UtcNow.AddMinutes(15), user.Adapt<UserDto>());
}
}

```

2.7. HIỆN THỰC HÓA ASP.NET CORE IDENTITY

ASP.NET Core Identity là framework quản lý user đầy đủ tính năng: tạo/xóa user, quản lý roles, xác nhận email, đặt lại mật khẩu, two-factor authentication, lockout và nhiều hơn nữa. Identity sử dụng IPasswordHasher (PBKDF2 mặc định) và tích hợp hoàn hảo với EF Core.

2.7.1 ApplicationUser — Mở rộng IdentityUser

```

C# CulinaryBlog.Domain/Entities/ApplicationUser.cs

using Microsoft.AspNetCore.Identity;

namespace CulinaryBlog.Domain.Entities;

// Mở rộng IdentityUser<string> để thêm các properties nghiệp vụ
// IdentityUser đã có sẵn: Id, Email, UserName, PasswordHash, EmailConfirmed,
...
public class ApplicationUser : IdentityUser
{
    public string FullName { get; set; } = string.Empty;
    public string? AvatarUrl { get; set; }
    public string? Bio { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    // Navigation properties
    public ICollection<Recipe> Recipes { get; set; } = [];
    public ICollection<Comment> Comments { get; set; } = [];
    public ICollection<RefreshToken> RefreshTokens { get; set; } = [];

    // Factory method – dùng cho đăng ký thông thường
    public static ApplicationUser Create(
        string fullName, string email, string username)
    {
        return new ApplicationUser
        {
            FullName = fullName,
            Email = email,
            UserName = username,
            // EmailConfirmed sẽ được set true sau khi user click link xác nhận
        };
    }
}

```

```

    }

    // Factory method – dùng khi đăng nhập qua Google/Social
    public static ApplicationUser CreateFromGoogle(
        string email, string fullName, string? avatarUrl)
    {
        return new ApplicationUser
        {
            FullName      = fullName,
            Email          = email,
            UserName       = email, // Dùng email làm username cho social login
            AvatarUrl      = avatarUrl,
            EmailConfirmed = true,  // Email đã được Google xác nhận
        };
    }
}

```

2.7.2 Identity Configuration và DI Setup

Text CulinaryBlog.Infrastructure/DependencyInjection.cs (Identity + JWT đầy đủ)

```

using CulinaryBlog.Application.Contracts.Services;
using CulinaryBlog.Domain.Entities;
using CulinaryBlog.Domain.Settings;
using CulinaryBlog.Infrastructure.Persistence;
using CulinaryBlog.Infrastructure.Services;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.IdentityModel.Tokens;
using System.Text;

namespace CulinaryBlog.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        // — Database —————
        services.AddDbContext<ApplicationDbContext>(options =>

options.UseNpgsql(configuration.GetConnectionString("DefaultConnection")));
        services.AddScoped<IApplicationDbContext, ApplicationDbContext>();

        // — ASP.NET Core Identity —————
        services
            .AddIdentity<ApplicationUser, IdentityRole>(options =>
            {
                // Chính sách mật khẩu
                options.Password.RequiredLength      = 8;
                options.Password.RequireDigit       = true;
                options.Password.RequireUppercase    = true;
                options.Password.RequireLowercase    = true;
            })
    }
}

```

```

        options.Password.RequireNonAlphanumeric = false;

        // Lockout: sau 5 lần sai → khóa 15 phút
        options.Lockout.MaxFailedAccessAttempts = 5;
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
        options.Lockout.AllowedForNewUsers = true;

        // Yêu cầu email duy nhất
        options.User.RequireUniqueEmail = true;
    })
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders(); // Cho email confirmation, password reset

    // — JWT Authentication —————
    var jwtSettings =
configuration.GetSection(JwtSettings.SectionName).Get<JwtSettings>(!);

services.Configure<JwtSettings>(configuration.GetSection(JwtSettings.SectionName));
services.AddScoped<IJwtService, JwtService>();

services
    .AddAuthentication(options =>
    {
        options.DefaultAuthenticateScheme =
JwtBearerDefaults.AuthenticationScheme;
        options.DefaultChallengeScheme =
JwtBearerDefaults.AuthenticationScheme;
    })
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwtSettings.Issuer,
            ValidAudience = jwtSettings.Audience,
            IssuerSigningKey = new SymmetricSecurityKey(
Encoding.UTF8.GetBytes(jwtSettings.Key)),
            // Không cho phép clock skew – token hết hạn là hết hạn
            ClockSkew = TimeSpan.Zero,
        };
    });

    // — Authorization —————
    services.AddAuthorizationBuilder()
        .AddPolicy("AdminOnly", p => p.RequireRole("Admin"))
        .AddPolicy("AuthorOrAdmin", p => p.RequireRole("Author", "Admin"))
        .AddPolicy("AuthenticatedUser", p => p.RequireAuthenticatedUser());

    services.AddSingleton<IAuthorizationHandler, RecipeAuthorizationHandler>();

    // — Rate Limiting —————
    services.AddRateLimiter(opts =>
    {
        opts.AddFixedWindowLimiter("login-policy", o =>

```

```

        {
            o.Window      = TimeSpan.FromMinutes(1);
            o.PermitLimit = 5;
            o.QueueLimit  = 0;
        });
        opts.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    });

    // — CORS —————
    services.AddCors(opts =>
    {
        opts.AddPolicy("NextJsPolicy", policy =>
            policy.WithOrigins("http://localhost:3000",
                               configuration["AllowedOrigins"] ?? string.Empty)
                .WithMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
                .WithHeaders("Authorization", "Content-Type", "Accept")
                .AllowCredentials());
    });

    return services;
}

```

2.8. CASE STUDY: HỆ THỐNG BẢO MẬT CULINARY BLOG

Case Study này kết hợp tất cả nội dung Chương 2: xây dựng hoàn chỉnh Auth Endpoints, từ đăng ký đến đăng nhập, refresh token, đăng xuất và bảo vệ Recipe endpoints với Owner Authorization. Đây là hệ thống bảo mật production-ready, không cần thêm gì để deploy.

Bước 1 — RegisterCommand: Đăng ký tài khoản

```

C#
CulinaryBlog.Application/Features/Auth/Commands/Register/RegisterCommandHandler.cs

public class RegisterCommandHandler : IRequestHandler<RegisterCommand,
AuthResponseDto>
{
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly IJwtService _jwtService;
    private readonly IApplicationDbContext _context;

    public async Task<AuthResponseDto> Handle(
        RegisterCommand command, CancellationToken ct)
    {
        // Kiểm tra email đã tồn tại chưa
        if (await _userManager.FindByEmailAsync(command.Email) is not null)
            throw new ConflictException($"Email '{command.Email}' đã được sử
dung.");

        // Tạo ApplicationUser qua factory method
        var user = ApplicationUser.Create(command.FullName, command.Email,
command.UserName);

        // Identity tự động hash mật khẩu với PBKDF2

```

```

var result = await _userManager.CreateAsync(user, command.Password);

if (!result.Succeeded)
{
    // Aggregate lỗi từ Identity (mật khẩu yếu, username trùng, ...)
    var errors = result.Errors.Select(e => e.Description);
    throw new ValidationException(
        new Dictionary<string, string> { ["Password"] =
errors.ToArray() });
}

// Gán role mặc định "Author"
await _userManager.AddToRoleAsync(user, "Author");

// Phát hành token ngay sau đăng ký (auto-Login)
var roles = await _userManager.GetRolesAsync(user);
var accessToken = _jwtService.GenerateAccessToken(user, roles);
var refreshToken = _jwtService.GenerateRefreshToken(user.Id);
_context.RefreshTokens.Add(refreshToken);
await _context.SaveChangesAsync(ct);

return new AuthResponseDto(accessToken, refreshToken.Token,
    DateTime.UtcNow.AddMinutes(15), user.Adapt<UserDto>());
}
}

```

Bước 2 — Auth Endpoints: Kết nối tất cả vào Minimal API

C# CulinaryBlog.API/Endpoints/AuthEndpoints.cs

```

using CulinaryBlog.Application.Features.Auth.Commands.Login;
using CulinaryBlog.Application.Features.Auth.Commands.Register;
using CulinaryBlog.Application.Features.Auth.Commands.RefreshToken;
using MediatR;

namespace CulinaryBlog.API.Endpoints;

public static class AuthEndpoints
{
    public static IEndpointRouteBuilder MapAuthEndpoints(
        this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/v1/auth")
            .WithTags("Authentication")
            .WithOpenApi();

        // POST /api/v1/auth/register
        group.MapPost("/register", async (
            RegisterCommand command,
            ISender sender,
            CancellationToken ct) =>
        {
            var result = await sender.Send(command, ct);
            return Results.Created($"{"/api/v1/users/{result.User.Id}", result);
        })
            .WithName("Register")
            .WithSummary("Đăng ký tài khoản mới")
    }
}

```

```

        .AllowAnonymous()
        .Produces<AuthResponseDto>(201)
        .ProducesProblem(400)
        .ProducesProblem(409);

// POST /api/v1/auth/Login
group.MapPost("/login", async (
    LoginCommand      command,
    ISender            sender,
    CancellationToken ct) =>
{
    var result = await sender.Send(command, ct);
    return Results.Ok(result);
})
.WithName("Login")
.WithSummary("Đăng nhập – nhận Access Token và Refresh Token")
.AllowAnonymous()
.RequireRateLimiting("login-policy") // Giới hạn 5 Lần/phút
.Produces<AuthResponseDto>()
.ProducesProblem(401);

// POST /api/v1/auth/refresh
group.MapPost("/refresh", async (
    RefreshTokenCommand command,
    ISender              sender,
    CancellationToken   ct) =>
{
    var result = await sender.Send(command, ct);
    return Results.Ok(result);
})
.WithName("RefreshToken")
.WithSummary("Làm mới Access Token bằng Refresh Token")
.AllowAnonymous()
.Produces<AuthResponseDto>()
.ProducesProblem(401);

// POST /api/v1/auth/Logout
group.MapPost("/logout", async (
    LogoutCommand      command,
    ISender            sender,
    CancellationToken ct) =>
{
    await sender.Send(command, ct);
    return Results.NoContent();
})
.WithName("Logout")
.WithSummary("Đăng xuất – thu hồi Refresh Token")
.RequireAuthorization(); // Phải đăng nhập mới được Logout

return app;
}
}

```

Bước 3 — Program.cs: Kết hợp tất cả middleware

Text CulinaryBlog.API/Program.cs (đầy đủ với bảo mật)

```
var builder = WebApplication.CreateBuilder(args);
```

```
// — Services
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);
builder.Services.AddOpenApi();

var app = builder.Build();

// — Middleware Pipeline (THỨ TỰ QUAN TRỌNG)
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();

// CORS phải đứng TRƯỚC Authentication
app.UseCors("NextJsPolicy");

// Rate Limiting
app.UseRateLimiter();

// Authentication TRƯỚC Authorization
app.UseAuthentication();
app.UseAuthorization();

// — Endpoints
app.MapAuthEndpoints();
app.MapCategoryEndpoints();
app.MapRecipeEndpoints();

app.Run();
```

Bước 4 — Kiểm thử với Scalar (HTTP request examples)

Text culinary-blog.http (REST Client file)

```
@baseUrl = https://localhost:5001/api/v1
@contentType = application/json

### Đăng ký tài khoản mới
POST {{baseUrl}}/auth/register
Content-Type: {{contentType}}

{
  "fullName": "Nguyễn Văn A",
  "email": "chef@culinaryblog.vn",
  "userName": "chef_nguyen",
  "password": "SecurePass123"
}

### Đăng nhập → lưu accessToken và refreshToken từ response
POST {{baseUrl}}/auth/login
Content-Type: {{contentType}}
```

```
{
  "email": "chef@culinaryblog.vn",
  "password": "SecurePass123"
}

### Tạo Recipe (yêu cầu JWT)
POST {{baseUrl}}/recipes
Content-Type: {{contentType}}
Authorization: Bearer {{accessToken}}

{
  "title": "Phở bò truyền thống",
  "description": "Công thức phở bò chuẩn vị Hà Nội",
  "categoryId": "...",
  "prepTimeMinutes": 30,
  "cookTimeMinutes": 180
}

### Refresh Token khi AccessToken hết hạn (sau 15 phút)
POST {{baseUrl}}/auth/refresh
Content-Type: {{contentType}}

{
  "accessToken": "{{expiredAccessToken}}",
  "refreshToken": "{{refreshToken}}"
}

### Test Rate Limiting: gửi 6 lần trong 1 phút → lần 6 phải nhận 429
POST {{baseUrl}}/auth/login
Content-Type: {{contentType}}

{
  "email": "wrong@email.com",
  "password": "WrongPassword"
}
```

CÂU HỎI THẢO LUẬN

1. Phân biệt Authentication và Authorization. Cho ví dụ: trong Culinary Blog, hành động nào chỉ cần Authentication và hành động nào cần cả Authorization?
2. Tại sao JWT là stateless? Ưu và nhược điểm của stateless authentication so với session-based authentication truyền thống là gì?
3. Giải thích "alg:none" attack trên JWT. Đoạn code trong `JwtService.GetPrincipalFromExpiredToken()` xử lý attack này như thế nào?
4. Token Rotation trong Refresh Token Architecture phát hiện và phản ứng thế nào khi Refresh Token bị đánh cắp? Mô tả chi tiết luồng xử lý.
5. PBKDF2 và bcrypt đều "chậm có chủ ý" — nhưng bcrypt còn có tính chất gì mà PBKDF2 không có? (Gợi ý: memory hardness)

6. Tại sao CORS không phải là biện pháp bảo mật API? Kẻ tấn công có thể bypass CORS không? Trong trường hợp nào?
7. Phân biệt Authorization Code Flow (với PKCE) và Implicit Flow trong OAuth 2.0. Tại sao IETF deprecate Implicit Flow?
8. Trong Resource-Based Authorization Handler, tại sao cần check cả role "Admin" ngoài việc check AuthorId? Nguyên tắc nào trong thiết kế hệ thống phân quyền điều này phản ánh?

BÀI TẬP THỰC HÀNH

Bài 1 — JWT và Identity cơ bản (Cơ bản)

- Implement đầy đủ RegisterCommand, LoginCommand và RefreshTokenCommand theo code mẫu trong chương.
- Tạo ApplicationUser, RefreshToken entity và cấu hình EF Core cho chúng.
- Cấu hình ASP.NET Core Identity với các ràng buộc mật khẩu: tối thiểu 8 ký tự, ít nhất 1 chữ hoa, 1 số.
- Test flow: Register → Login → nhận JWT → gửi request với JWT → verify response.
- Chạy dotnet user-secrets để lưu JwtSettings:Key an toàn (không commit vào Git).

Bài 2 — Authorization nâng cao (Trung bình)

- Implement LogoutCommand: tìm và thu hồi Refresh Token của user hiện tại.
- Implement RecipeAuthorizationHandler đầy đủ: Reader chỉ đọc, Author chỉ sửa/xóa recipe của mình, Admin toàn quyền.
- Thêm endpoint GET /api/v1/auth/me trả về thông tin user đang đăng nhập (lấy từ JWT claims, không query database).
- Implement "Revoke all sessions" endpoint: khi user đổi mật khẩu, thu hồi tất cả Refresh Token cũ.
- Viết unit tests cho RecipeAuthorizationHandler với 3 test cases: Author của recipe, Author khác, Admin.

Bài 3 — Rate Limiting, CORS và Social Login (Nâng cao)

- Cấu hình Rate Limiting: Login endpoint 5 req/phút, Register endpoint 3 req/phút, API chung 200 req/phút.
- Thêm custom response header "X-RateLimit-Remaining" cho các endpoint có Rate Limiting.
- Tạo Google Cloud Console project, lấy Client ID và Secret, implement Google Social Login.
- Xử lý trường hợp: user đăng ký thủ công với email A, sau đó đăng nhập Google với cùng email A — merge accounts hay báo lỗi?
- Viết integration tests với TestContainers cho toàn bộ auth flow (register → login → refresh → logout).

TÓM TẮT NỘI DUNG CỐT LÕI

Khái niệm	Mô tả ngắn gọn	Implementation trong Culinary Blog
PBKDF2	Key derivation function "chậm có chủ ý" — chống brute-force	ASP.NET Core Identity mặc định, 600k iterations
JWT Structure	Header.Payload.Signature — Base64Url, chỉ Signature được ký	HS256, claims: sub/email/role/jti
Access Token	Short-lived JWT (15 phút) — xác thực mỗi request	Bearer token trong Authorization header
Refresh Token	Long-lived random bytes (7 ngày) — lưu DB, dùng để refresh	RefreshToken entity + IsUsed/IsRevoked flags
Token Rotation	Mỗi lần refresh: tạo RT mới, đánh dấu RT cũ IsUsed	RefreshTokenCommandHandler.MarkUsed()
RBAC	Phân quyền theo Role trong JWT claims	.RequireRole("Admin"), .RequireAuthorization()
Resource-Based	Phân quyền theo thuộc tính resource cụ thể	RecipeAuthorizationHandler checks AuthorId
Rate Limiting	Giới hạn tần suất request — chống brute-force	Fixed Window 5 req/phút cho /auth/login
CORS	Server cho phép browser cross-origin requests	WithOrigins("localhost:3000"), AllowCredentials
OAuth 2.0 + PKCE	Ủy quyền bên thứ ba — Authorization Code Flow	AddGoogle(), callback xử lý ID Token

Từ khóa quan trọng:

PBKDF2, [PasswordHasherOptions](#), JWT (Header/Payload/Signature), [HS256](#) vs [RS256](#), Claims, Access Token, Refresh Token, Token Rotation, [RefreshToken.MarkUsed\(\)](#), RBAC, Policy-Based Authorization, Resource-Based Authorization, [IAuthorizationService.AuthorizeAsync\(\)](#), Fixed Window Rate Limiting, CORS, Same-Origin Policy, OAuth 2.0, PKCE, [AddAuthentication\(\)](#), [AddJwtBearer\(\)](#), [UseAuthentication\(\)](#) / [UseAuthorization\(\)](#)

TÀI LIỆU THAM KHẢO MỞ RỘNG

Tiêu chuẩn và RFC:

- RFC 7519 — JSON Web Token (JWT):
<https://datatracker.ietf.org/doc/html/rfc7519>

- RFC 6749 — OAuth 2.0 Authorization Framework:
<https://datatracker.ietf.org/doc/html/rfc6749>
- RFC 7636 — PKCE for OAuth 2.0 Public Clients:
<https://datatracker.ietf.org/doc/html/rfc7636>
- OWASP Authentication Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP JWT Security:
https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

Tài liệu kỹ thuật chính thức:

- Microsoft Learn — ASP.NET Core Identity:
<https://learn.microsoft.com/aspnet/core/security/authentication/identity>
- Microsoft Learn — JWT Bearer Authentication:
<https://learn.microsoft.com/aspnet/core/security/authentication/jwt-authn>
- Microsoft Learn — Resource-based Authorization:
<https://learn.microsoft.com/aspnet/core/security/authorization/resourcebased>
- Microsoft Learn — Rate Limiting in .NET:
<https://learn.microsoft.com/aspnet/core/performance/rate-limit>

Bài viết chuyên sâu:

- PortSwigger Web Security Academy — JWT Attacks (hands-on labs):
<https://portswigger.net/web-security/jwt>
- Auth0 Blog — Refresh Token Rotation:
<https://auth0.com/docs/secure/tokens/refresh-tokens/refresh-token-rotation>
- Troy Hunt — Passwords Evolved (why hashing matters):
<https://www.troyhunt.com/passwords-evolved-authentication-guidance-for-the-modern-era/>
- NIST Special Publication 800-63B — Digital Identity Guidelines (tiêu chuẩn chính phủ Mỹ về authentication): <https://pages.nist.gov/800-63-3/sp800-63b.html>

Hết Chương 2